Test Strategy – Automated Testing Implementation

Below is a **Confluence-ready Test Strategy document** that continues from your **Risk-Based Testing (RBT) framework page**.

It is written as a **professional QA Test Strategy**, aligned with the README you shared, but formatted as a **clear engineering document** suitable for demoing to **engineering management**.

You can **copy-paste this directly into Confluence**.

Test Strategy – Automated Testing Implementation

Project: Corto QE Automation Framework

---

1. Objective

This document describes the **test strategy and implementation approach** used to build the automated testing framework for the Corto QE project.

The strategy follows the **Risk-Based Testing (RBT)** methodology documented in the companion Confluence page:

Risk-Based Testing Framework

Testing effort is prioritized using **Risk Priority Number (RPN)** scoring to ensure that automation focuses first on **the most critical and failure-prone areas of the system**.

The strategy covers:

- Test prioritization approach
- Automation scope
- Technology stack selection
- Tooling decisions
- Test architecture
- Execution workflow

---

2. System Under Test

The automation framework validates two independent applications:

Application	Type	URL
DemoQA Book Store	Web UI	demosite
Restful Booker	REST API	https://restful-booker.herokuapp.com

Both applications are tested within a **single unified automation framework**.

---

3. Risk-Based Test Prioritization

Following the RBT methodology, each identified scenario was assigned:

- **Likelihood (L)** – probability of failure
- **Impact (I)** – severity if failure occurs

Risk Priority Number:

RPN = Likelihood × Impact

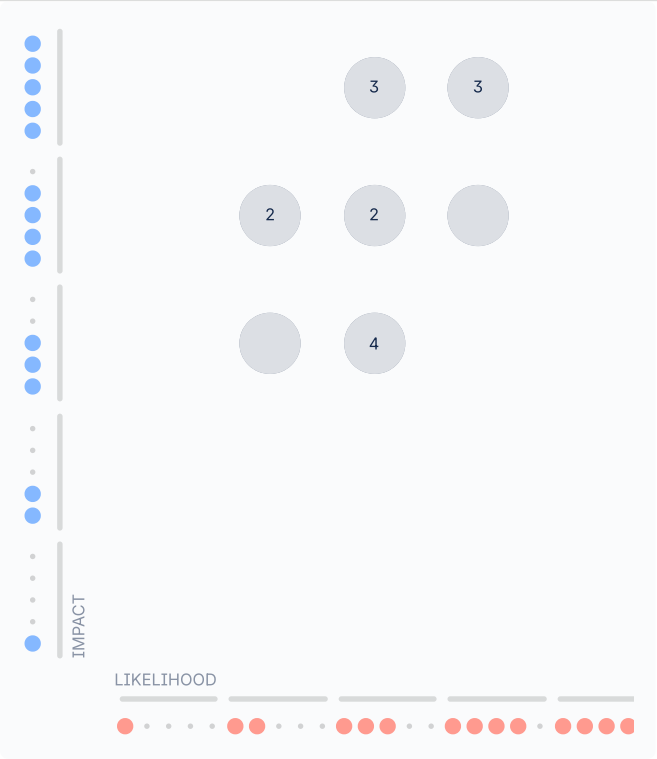
Scenarios were then ranked by **RPN score**.

Automation was implemented **in descending RPN order**.

---

4. Scenario Prioritization

🖼️ FrontEnd | Automation Priority: DemoQA Book Store UI Impact vs Effort



Book Store page loads and displays all 8 b...
Likelihood
Impact

Book table shows correct columns (Image...
Likelihood
Impact

Each book row displays correct title, autho...
Likelihood
Impact

Search filters books by title keyword
Likelihood
Impact

Search filters books by author name
Likelihood
Impact

Automation order followed the same sequence.

This ensures that **if execution time or resources are limited, the highest-risk areas are still fully covered.**

---

5. Automation Scope

Included in Automation

API Testing

- Authentication
- Booking creation
- Booking retrieval
- Booking update
- Booking deletion
- End-to-end CRUD flow
- Negative validation scenarios

UI Testing

- Book listing page
- Book search
- Book details view
- Table rendering

Out of Scope (For MVP)

- Cross-browser testing
- Mobile viewport testing
- Visual regression testing
- Performance testing

These are listed in the **Future Enhancements** section.

---

6. Technology Stack Selection

Layer	Technology	Rationale

Language	TypeScript / Node.js	Strong typing, excellent Playwright support
Test Framework	Playwright	Supports both UI and API testing
Package Manager	pnpm	Faster dependency installs, disk efficiency
CI/CD	GitHub Actions	Native GitHub integration
Version Control	GitHub	Standard repository management
Manual API Exploration	Postman	Rapid API discovery and debugging

---

7. Why Playwright Was Chosen

Although Playwright is widely known for UI automation, it also provides a **first-class API testing capability**.

The key advantages for this project were:

Unified Toolchain

A single framework was used for:

- UI testing
- API testing
- reporting
- CI execution

This eliminates the need to maintain **multiple frameworks**.

Built-in API Client

Playwright provides a built-in **HTTP request context** that supports:

- full REST calls
- authentication
- headers
- assertions
- response validation

Shared Test Runner

Both UI and API tests run through:

```
Playwright Test Runner
```

Benefits:

- unified reporting
- shared configuration
- single CI pipeline

---

8. Test Architecture

The framework follows several design patterns to ensure maintainability and scalability.

Page Object Model (UI)

Each UI page is represented by a **Page Object**.

Responsibilities:

Layer	Responsibility
Page Object	locators and UI interactions
Test Spec	assertions and validation

Example structure:

```
src/ui/pages
   base.page.ts
   bookStore.page.ts
   bookDetails.page.ts
```

Benefits:

- reusable interactions
- clean test code
- maintainable locators

API Client Abstraction

API tests use a thin wrapper around Playwright's request context.

File: `src/helpers/api.client.ts`

Responsibilities:

- authentication token handling
- standardized request method
- header management

Example abstraction: `apiClient.send(method, path, payload)`

Benefits:

- avoids repetitive boilerplate
- simplifies test specs
- centralizes request logic

---

9. Data-Driven Testing

Test data is separated from test logic.

Structure:

```
src/ui/data
src/api/data
```

Benefits:

- reusable datasets
- easier negative testing
- easier scenario expansion

Adding new test cases requires only **updating the data files**.

---

10. Test Tagging Strategy

Tests are categorized using **tags**.

Example:

```
@negative
@api
@ui
```

This allows selective execution.

Example commands:

Run negative tests only

```
pnpm test-negative

```

Run API tests only

```
pnpm test-api

```

---

11. Execution Strategy

The test suite supports multiple execution modes.

Command	Purpose
pnpm test	Run all tests
pnpm test-ui	Run UI tests only
pnpm test-api	Run API tests only
pnpm test-negative	Run negative scenarios
pnpm test-open	Interactive Playwright UI mode

---

12. CI/CD Pipeline Strategy

Automation is integrated with **GitHub Actions**.

The pipeline performs:

- dependency installation
- browser setup
- parallel test execution
- report artifact upload

Parallel Execution

UI and API suites run as **separate jobs**.

Benefits:

- faster pipeline runtime

- independent failure visibility

CI Caching Optimization

Pipeline performance is improved using:

Cache	Benefit
pnpm dependency cache	Avoid repeated installs
Playwright browser cache	Skip large browser downloads

This reduces CI runtime significantly.

---

13. Reporting

After each test run, Playwright generates a **HTML report**.

View report locally:

```
pnpm exec playwright show-report

```

Reports contain:

- pass/fail status
- screenshots
- traces
- step-by-step execution

---

14. Defects Identified

During automation development, **three genuine defects** were discovered in the Restful Booker API.

Example defect:

Endpoint	Issue
POST /booking	Server crashes when numeric values are passed in name fields

Expected

```
400 Bad Request

```

Actual

```
500 Internal Server Error

```

Impact

Missing input validation causing server crash.

All defects were logged in **Jira with reproduction steps and impact analysis**.

---

15. Traceability

Full traceability exists between:

```
Risk Matrix
   ↓
Jira Scenario Ticket
   ↓
Git Branch
   ↓
Pull Request
   ↓
Automation Test

```

This ensures every automated test corresponds to a **documented scenario and risk assessment**.

---

16. Future Enhancements

The following improvements are planned beyond the MVP implementation.

RPN-Based Test Selection

Tests can be tagged with risk level:

```
@high
@medium
@low

```

CI can then execute:

- High-risk tests on every PR

- Full suite on nightly runs

Cross-Browser Testing

Add Playwright projects for:

- Chromium
- Firefox
- WebKit

Visual Regression Testing

Critical UI flows can use screenshot comparisons:

```
toHaveScreenshot()

```

Scheduled Regression Runs

Nightly CI execution ensures system stability even without code changes.

Slack Notifications

Test failures can be pushed directly to engineering Slack channels.

---

17. Key Outcome

This strategy ensures that automation:

- focuses on **highest-risk areas first**
- provides **fast feedback through CI**
- maintains **clean, scalable architecture**
- supports **future expansion**

By aligning testing effort with **Risk-Based Testing principles**, the framework maximizes defect detection while keeping the automation suite efficient and maintainable.